

CSCE 611 OPERATING SYSTEMS · MODULE 06 · LECTURE NOTES

Memory Allocation

Handing out memory, and the fragmentation that follows.

MODULE OUTCOMES

- Explain internal and external fragmentation.
- Compare first fit, best fit, and segregated free lists.
- Explain buddy and slab allocation and where each is used.
- Explain why allocator performance depends on usage pattern.
- Explain arena and pool allocation and when to use them.

1 What makes allocation hard

A general-purpose allocator must satisfy requests of arbitrary size, arriving in arbitrary order, with arbitrary and unknown lifetimes — and it must be fast, because malloc appears in the profile of almost every program.

THE CONSTRAINT THAT MAKES IT GENUINELY HARD

An allocation can never be moved. The caller holds a raw pointer, possibly copied into other structures, and the allocator has no way to find and update those copies.

A garbage-collected runtime *can* move objects, because it knows where every reference lives. That is precisely why compacting collectors can eliminate fragmentation entirely and C allocators cannot.

Everything difficult about malloc follows from this one restriction.

1.1 Two fragmentations

	Internal fragmentation	External fragmentation
What	Wasted space <i>inside</i> an allocated block.	Wasted space <i>between</i> allocated blocks.
Example	A 33-byte request served by a 48-byte size class wastes 15 bytes.	Several free blocks totalling 10 KB, none individually large enough for a 4 KB request.
Cause	Size classes, alignment requirements, per-block headers.	Interleaved allocation and freeing of mixed sizes with mixed lifetimes.
Predictability	Bounded and computable from the size classes.	Depends on the entire allocation history. Essentially unpredictable.

	Internal fragmentation	External fragmentation
Fix	Finer size classes, at the cost of more lists.	Requires moving allocations — which is forbidden. Can only be mitigated.

ALLOCATION IS AN ONLINE PROBLEM

Given the complete sequence of future requests, optimal placement could be computed. The allocator does not have it: each decision is made blind, and a placement that is fine now may strand memory later.

This is an **online algorithm** in the sense of CSCE 629, judged by its competitive ratio against the offline optimum, and no online allocator is optimal across all workloads.

That is why many allocators exist and why 'which allocator is fastest' has no answer without naming the program. Measure on your workload.

2 Placement policies

Policy	Rule	Behaviour in practice
First fit	Scan from the start; take the first block large enough.	Fast in the common case. Tends to fragment the front of the list, which then has to be scanned repeatedly.
Next fit	Like first fit, but resume scanning where the last search ended.	Spreads allocations across the heap. Usually worse, because it destroys locality.
Best fit	Scan everything; take the smallest block that fits.	Minimises per-allocation waste and produces many unusably small remainders. Slow, because it scans the whole list.
Worst fit	Take the largest block.	Leaves large usable remainders and consumes the big blocks you will later need. Rarely used.
Segregated free lists	One list per size class; take from the matching list.	O(1) . What real allocators do.

BEST FIT IS USUALLY WORSE THAN FIRST FIT

Best fit sounds obviously correct: minimise the waste on every allocation.

The consequence is that every leftover is as small as possible — and a very small leftover is useless. The heap fills with thousands of fragments too small to satisfy any request, which is external fragmentation in its purest form.

First fit leaves larger, more reusable remainders and scans far less. It is typically both faster and less fragmenting.

This is the same lesson as greedy algorithms in CSCE 629 Module 09: a locally optimal choice is not globally optimal, and the plausible rule needs an argument rather than an intuition.

3 How real allocators work

3.1 Segregated free lists

Maintain a separate free list for each **size class** — 16, 32, 48, 64, 80 bytes, and so on, often with geometric spacing above some threshold.

- **Allocate:** round the request up to the next class, pop the head of that list. $O(1)$, a handful of instructions.
- **Free:** push onto the list for that block's class. $O(1)$.
- **Cost:** internal fragmentation bounded by the class spacing — at most 15 bytes with 16-byte spacing.
- **Large allocations bypass the mechanism** and go straight to `mmap`. This is why freeing a very large allocation often returns memory to the operating system immediately, while freeing many small ones usually does not.

3.2 Specialised allocators

Allocator	Mechanism	Where used
Buddy	Memory is split into power-of-two blocks. A block is halved repeatedly until the right size is reached; on free, a block is merged with its 'buddy' if that is also free.	Kernel physical page allocation. Coalescing is $O(\log n)$ and finding the buddy is a single XOR.
Slab	A cache per <i>object type</i> , holding pre-initialised objects. Allocation returns one that is already constructed.	Kernel objects created and destroyed constantly: inodes, file structures, process control blocks. Avoids repeated initialisation and keeps objects cache-aligned.
Arena / region	Bump a pointer. Free everything at once by resetting it.	Per-request, per-frame, per-parse. See below.
Pool	Fixed-size objects with a simple free list.	Particles, tree nodes, entities — anywhere objects are homogeneous.
Stack	Strictly LIFO allocation and deallocation.	Scratch buffers within a scope. Trivially fast and trivially constrained.

ARENA ALLOCATION IS THE LARGEST EASY WIN AVAILABLE

Allocate one large block up front. Satisfy each request by returning the current pointer and advancing it. Release everything in one operation by resetting the pointer to the start.

Allocation is three instructions. Deallocation is one. There is no free list, no coalescing, no search, no per-allocation header, and no fragmentation of any kind.

It applies whenever a set of allocations **shares a lifetime**: everything needed to serve one HTTP request, to render one frame, to parse one document, to compile one translation unit. That describes a very large fraction of real allocation.

Replacing general-purpose malloc with an arena in a hot path is routinely a several-fold speedup, and the surrounding code usually becomes *simpler*, because individual objects no longer need to be freed and ownership questions disappear.

4 Allocators on multicore

A single global heap protected by one lock is precisely the contended-atomic scalability failure of CSCE 614 Module 11: every thread competes for one cache line, and adding cores makes it worse.

Modern allocators therefore give each thread a **private cache** of free blocks, refilled in batches from a shared pool. The great majority of allocations and frees then touch no shared state at all.

The complication is that a block allocated on thread A may be freed on thread B. Allocators handle this with per-thread remote free lists, batching freed blocks back to their owning thread periodically rather than synchronising on each one. This is the design behind tcmalloc, jemalloc, and mimalloc, and it is why they substantially outperform older system allocators on threaded workloads.

5 Choosing

Situation	Recommendation
General purpose, unknown allocation pattern.	Use the system allocator. Measure before replacing it — modern ones are good.
Many threads allocating heavily.	jemalloc, tcmalloc, or mimalloc. Often a large improvement for a one-line change.
A set of allocations sharing a lifetime.	An arena. Usually the single biggest win, and it simplifies ownership.
Many objects of one type.	A pool or slab allocator.
Hard latency bounds — audio, control loops, game frame loops.	Pre-allocate everything. Never call malloc in the hot path: a general allocator's worst case is unbounded, and the occasional slow allocation is exactly what blows a frame budget.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OSTEP — Chapter 17 (Free-Space Management)	Placement policies, coalescing, buddy and slab allocation, with worked examples.
Wilson et al. — Dynamic Storage Allocation: A Survey and Critical Review (free)	The definitive survey, and unusually sharp about how much allocator research measured the wrong things.
Bonwick — The Slab Allocator (free)	The original slab paper. Short, and the reasoning about object construction cost is still relevant.
mimalloc technical report (free)	A modern multithreaded allocator explained by its authors, including the remote-free problem.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a simple free-list allocator with first fit, best fit, and worst fit. Run a mixed workload through each and measure fragmentation and allocation time.
- 2 Measure external fragmentation directly: allocate and free in a pattern designed to strand memory, then report total free bytes against the largest contiguous free block.
- 3 Implement segregated free lists and compare allocation time against your first-fit implementation. Measure the internal fragmentation you traded for it.
- 4 Implement an arena allocator. Replace malloc with it in a parser or a per-frame workload and measure the speedup.
- 5 Implement a buddy allocator and verify that freeing merges blocks correctly. Confirm that finding a buddy is a single XOR.
- 6 Measure the system allocator against jemalloc or mimalloc on a workload with eight threads allocating concurrently. Report the scaling of each.
- 7 Write a program that allocates a 1 GB block, frees it, and checks whether resident memory returns to the OS. Repeat with a million small allocations and explain the difference.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why can a C allocator not move allocations, and what does that make impossible?
- 2 Distinguish internal from external fragmentation, including which is predictable.
- 3 Why is allocation an online problem, and what follows about choosing an allocator?
- 4 Explain why best fit is usually worse than first fit.
- 5 How do segregated free lists achieve $O(1)$ allocation, and what do they trade for it?
- 6 What is arena allocation, when does it apply, and why does it also simplify code?
- 7 Why do modern allocators maintain per-thread caches, and what complication does that introduce?